

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Практическая работа № 4

Выполнил:

Смирнова Карина

К3341

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Технический дизайн микросервисной архитектуры

Задание:

1. Разделите ваше монолитное бэкэнд-приложение на микросервисы, придерживаясь концепции database-per-service
2. Спроектируйте микросервисную архитектуру: отразите взаимосвязь между микросервисами, опишите способы их взаимодействия друг с другом
3. Спроектируйте разделение вашей БД на обособленные БД
4. Спроектируйте и опишите эндпоинты, необходимые для обеспечения межсервисного взаимодействия в формате спецификации OpenAPI
5. Опишите варианты запросов и ответов, задокументируйте возможные ошибки
6. В результате вы должны сформировать документ, в котором будут описаны конкретные требования и шаги для достижения разделения вашего монолитного бэкэнд-приложения на отдельные микросервисы

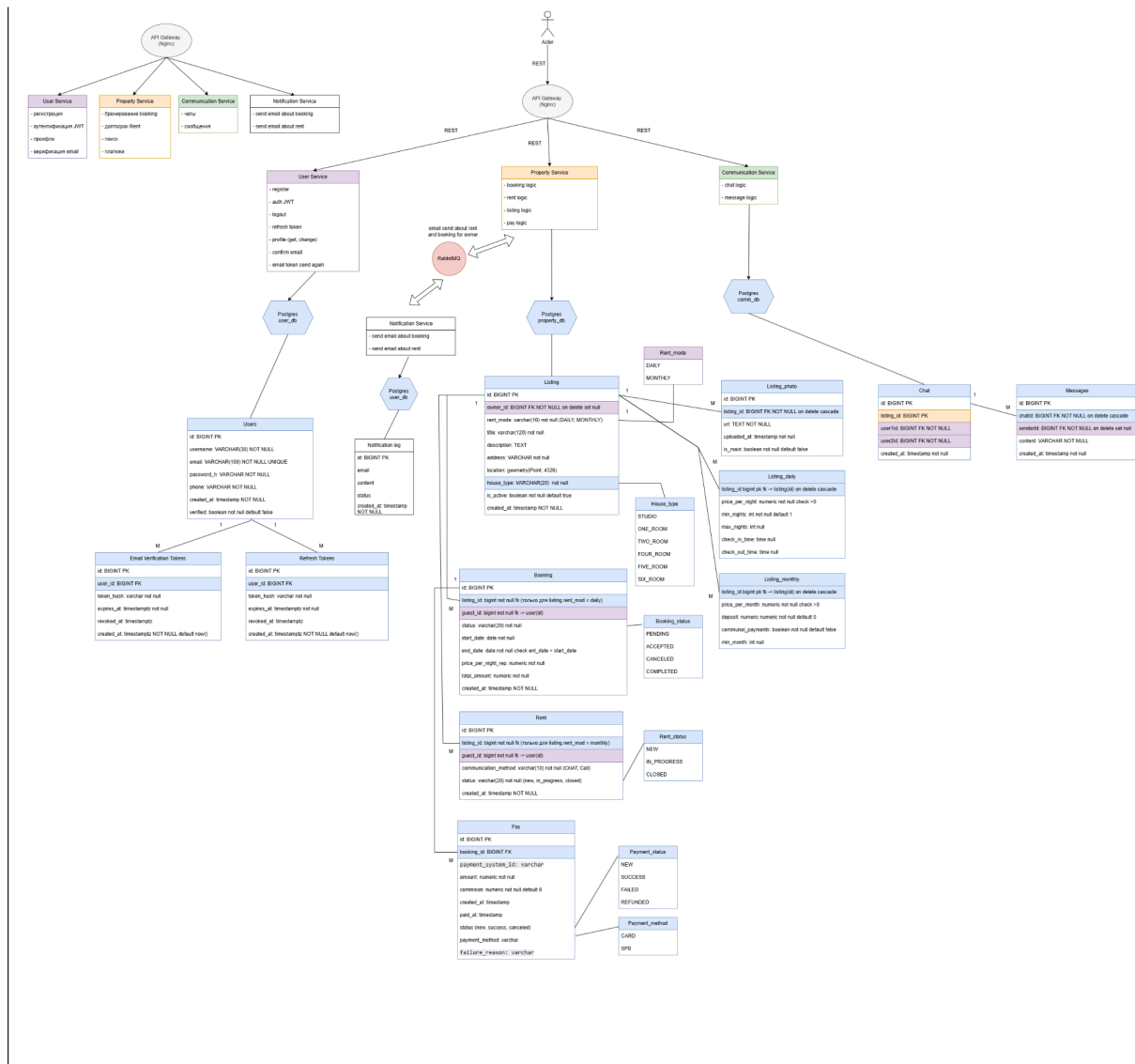
Ход работы

1. Общая архитектура системы

Система переведена с монолитной архитектуры на микросервисную.

- **API Gateway (Nginx):** Выступает в качестве единой точки входа для внешних клиентов (клиентских приложений). Он отвечает за маршрутизацию входящих HTTP-запросов на соответствующие микросервисы, а также может брать на себя функции SSL-терминации и первичной фильтрации трафика.
- **Межсервисное взаимодействие:**
 - **Синхронное (REST API):** Используется там, где необходим немедленный ответ (например, аутентификация, создание чата привязанного к объявлению, получение профиля).
 - **Асинхронное (Event Bus / RabbitMQ):** Используется для отправки уведомлений. Сервисы публикуют события в брокер сообщений, что исключает жесткую связанность (loose coupling) и защищает систему от задержек при отправке писем.

- **Изолированные БД** - упрощения dev-среды они могут физически лежать в одном Postgres-контейнере, но логически разделены.



2. Описание микросервисов и их API

User Service (Сервис пользователей)

Отвечает за управление учетными данными, профилями пользователей, сессиями и безопасностью.

- **База данных:** Изолированная БД user_db (PostgreSQL). Хранит основную информацию о пользователях (users), токены верификации email (Email Verification Tokens) и токены обновления сессий (Refresh Tokens).
- **Ключевой функционал API:**

- POST /auth/register и POST /auth/login — регистрация и аутентификация с выдачей пары JWT (Access/Refresh).
- POST /auth/refresh и POST /auth/logout — управление жизненным циклом сессии.
- POST /users/email/confirm — подтверждение почты.
- **Внутреннее API:** * POST /auth/validate — эндпоинт для валидации токенов другими микросервисами.
 - GET /internal/users/{userId} — предоставление публичной информации о пользователе для Property/Communication сервисов.

Auth Авторизация и верификация пользователя

POST	/auth/register	Регистрация пользователя	▼
POST	/auth/login	Вход пользователя	▼
POST	/auth/refresh	Обновление токенов	▼
POST	/auth/logout	Логаут пользователя	▼
POST	/users/email/confirm	Подтверждение email по токenu	▼
POST	/auth/validate	Валидация access token (для других сервисов)	▼

Users Профиль и настройки текущего пользователя

GET	/users/me	Получить свой профиль	▼
PATCH	/users/me	Обновить свой профиль	▼
POST	/users/me/email/verify	Повторная отправка email	▼
GET	/internal/users/{userId}	Получение публичного профиля пользователя для других сервисов	▼

Property Service (Сервис недвижимости и бронирования)

Центральный бизнес-блок системы. Отвечает за размещение объявлений, ведение каталога недвижимости, логику краткосрочного бронирования и долгосрочной аренды, а также проведение платежей.

- **База данных:** Изолированная БД property_db (PostgreSQL). Включает таблицы объявлений (Listing), типов жилья (House_type), цен/условий (Listing_daily, Listing_monthly), фото, а также логику процессов: Booking (краткосрочно), Rent (долгосрочно) и Pay (транзакции и интеграция с платежными шлюзами).
- **Особенности интеграции:** При инициации чатов по объекту обращается к Communication Service через REST.

- При успешной оплате бронирования или создании заявки на аренду генерирует события BookingPaid и RentCreated в RabbitMQ.

Listings Объявления и фотографии



GET	/listings	Поиск объявлений	▼
POST	/listings	Создать объявление	▼
GET	/listings/{listingId}	Детали объявления	▼
PATCH	/listings/{listingId}	Обновить объявление	▼
POST	/listings/{listingId}/deactivate	Деактивировать объявление	▼
POST	/listings/{listingId}/activate	Активировать объявление	▼
GET	/listings/{listingId}/photos	Список фото	▼
POST	/listings/{listingId}/photos	Добавить фото	▼
DELETE	/listings/{listingId}/photos/{photoId}	Удалить фото	▼
PATCH	/listings/{listingId}/photos/{photoId}	Обновить фото	▼

Bookings Бронирования жилья



GET	/bookings	Список бронирований	▼
POST	/bookings	Создать бронирование	▼
GET	/bookings/{bookingId}	Детали бронирования	▼
POST	/bookings/{bookingId}/complete	Завершить бронирование	▼
POST	/bookings/{bookingId}/cancel	Отменить бронирование	▼

Rents Заявки на аренду			^
GET	/rents	Список заявок	▼
POST	/rents	Создать заявку	▼
GET	/rents/{rentId}	Детали заявки	▼
PATCH	/rents/{rentId}	Обновить заявку	▼
POST	/rents/{rentId}/close	Закрыть заявку	▼
Payments Оплаты и чеки			^
GET	/bookings/{bookingId}/payments	Список оплат по бронированию	▼
POST	/bookings/{bookingId}/payments	Создать оплату по бронированию	▼
POST	/payments/{paymentId}/retry	Повторить оплату	▼
GET	/payments/{paymentId}	Детали оплаты	▼
GET	/payments/{paymentId}/receipt	Получить чек	▼

Communication Service (Сервис коммуникаций)

Обеспечивает общение между арендаторами и арендодателями внутри платформы.

- **База данных:** Изолированная БД comm_db (PostgreSQL). Хранит сущности чатов (Chat) и историю сообщений (Messages). Чаты жестко привязаны к listing_id (объекту недвижимости).
- **Ключевой функционал API:**
 - GET /chats и POST /chats — получение списка чатов текущего пользователя и создание нового чата.
 - GET /chats/{chatId}/messages и POST /chats/{chatId}/messages — чтение и отправка сообщений.
 - **Внутреннее API (POST /internal/chats):** Позволяет *Property Service* автоматически инициировать чат между пользователями (например, при подаче заявки на бронирование).

Chats Чаты



GET	/chats	Список чатов для пользователя	▼
POST	/chats	Создать чат	▼
DELETE	/chats/{chatId}	Удалить чат	▼
POST	/internal/chats	Внутренний эндпоинт для создания чата между пользователями (Property Service → Communication)	▼

Messages Сообщения



GET	/chats/{chatId}/messages	Список сообщений в чате	▼
POST	/chats/{chatId}/messages	Отправить сообщение в чат	▼

Schemas



CreateChatRequest >

SendMessageRequest >

Notification Service (Сервис уведомлений)

Вспомогательный сервис, работающий в фоновом режиме. Он изолирует тяжелые операции по отправке email/push-уведомлений от основной бизнес-логики.

- **База данных:** Изолированная БД user_db (на схеме помечена как Notification log / user_db, используется для логирования фактов отправки).
- **Принцип работы:** Не предоставляет публичного REST API для пользователей (доступен только внутренний /internal/notifications/health). Сервис подписан на очередь в **RabbitMQ** и асинхронно обрабатывает события (Events):
 - BookingPaid — отправляет владельцу (Owner) email о том, что его объект успешно забронирован на определенные даты.
 - RentCreated — отправляет владельцу email о наличии новой заявки на долгосрочную аренду.

Notification Service API 1.0.0 OAS3

<https://raw.githubusercontent.com/asynccapi/spec/v2.5.0/examples/streetlights-kafka.yml>

Notification Service слушает события из RabbitMQ (EventBus). Ниже указаны контракты событий, которые сервис обрабатывает. Это **не публичные REST API**, а описание внутренних событийных сообщений.

Servers

▾

Notifications Monitoring endpoints

GET Health check ▾

3. Схема взаимодействия компонентов (Data Flow)

Для отчета полезно зафиксировать два основных сквозных сценария:

1. Создание и оплата бронирования (Краткосрочно):

- Пользователь отправляет запрос на бронирование в Property Service.
- Property Service проходит этап верификации токена через User Service (как описано выше).
- После успешной проверки Property Service проверяет доступность дат, создает запись Booking в статусе PENDING и инициирует платеж через таблицу Pay.
- После успешного списания средств (Payment_status = SUCCESS) Property Service публикует асинхронное событие BookingPaid в **RabbitMQ**.
- Notification Service мгновенно вычитывает это событие из брокера очередей и отправляет email-уведомление владельцу жилья ("ваш объект арендован").

2. Коммуникация по объявлению:

- Пользователь хочет уточнить детали и открывает чат. Property Service (после прохождения верификации) через внутренний эндпоинт POST /internal/chats связывается с Communication Service, автоматически создавая защищенный чат между user1Id, user2Id и привязывая его к конкретному listingId.

- Дальнейшая переписка идет напрямую через Communication Service, где каждый запрос на отправку/получение сообщений также проходит валидацию токена через User Service.

Вывод

В рамках работы осуществлен переход от монолитной архитектуры к микросервисной. Были спроектированы и реализованы независимые сервисы: User Service, Property Service, Communication Service и Notification Service, каждый из которых отвечает только за свой домен и использует отдельную базу данных. Для межсерверного взаимодействия реализованы как синхронные REST API (для быстрого обмена критичными данными), так и асинхронный Event Bus на RabbitMQ (для уведомлений и слабой связанности сервисов).

Вся функциональность исходного монолита была полностью сохранена и разбита на соответствующие микросервисы. Дополнительно в архитектуру внедрен Notification Service, который обеспечивает надежную и асинхронную отправку email-уведомлений владельцам недвижимости о событиях бронирования и новых заявках на аренду.